EXPERIMENT NO. 5

Hands-on Solidity Programming Assignments for Creating Smart Contracts

Aim

To gain hands-on experience with Solidity programming by creating, compiling, deploying, and executing smart contracts using Remix IDE.

Theory

1. Solidity and Smart Contracts

Solidity is a high-level, statically typed programming language used for developing smart contracts on Ethereum-compatible blockchain platforms. A smart contract is a program stored on the blockchain that executes predefined logic when its functions are called. Remix IDE provides an environment to write, compile, deploy, and test Solidity contracts. Remix VM provides a simulated blockchain environment for practical experimentation.

2. Primitive Data Types and Variables

Solidity provides basic data types such as uint for unsigned integers, int for signed integers, bool for true/false values, address for Ethereum addresses, string for text, and bytes for binary data. Variables may be state variables, local variables, or special global values such as msg.sender, msg.value, and block.timestamp.

- uint/int – store unsigned/signed integer values.
- bool – stores true or false.
- address – stores a 20-byte Ethereum address.
- string/bytes – store text or binary data.
- State variables are stored permanently on the blockchain.

3. Functions – pure and view

Functions contain the executable logic of a smart contract. A view function can read state variables but cannot modify blockchain state. A pure function cannot read or modify contract state and performs calculations only from its inputs and internal variables.

Example:

```
function getNumber() public view returns (uint) { return number; }

function add(uint a, uint b) public pure returns (uint) { return a + b; }
```

4. Inputs and Outputs to Functions

Functions can accept input parameters and return one or more output values. Inputs allow users or other contracts to pass data, while return values provide the result of the operation.

5. Visibility, Modifiers and Constructors
Visibility controls who can access a function. public functions can be called internally and externally; private functions are accessible only within the same contract; internal functions are accessible within the contract and derived contracts; external functions are intended to be called from outside the contract.

Modifiers are reusable code blocks used to add conditions to functions, commonly for access control. A constructor executes only once when a contract is deployed and is generally used to initialize values such as the contract owner.

6. Control Flow – if-else and Loops
Solidity supports conditional statements such as if-else for decision making and loops such as for, while, and do-while for repeated execution. Loops should be used carefully because every operation executed in a blockchain transaction contributes to gas consumption.

7. Data Structures
Arrays store ordered collections of values. Mappings store key-value pairs and are useful for fast lookups. Structs group related values into a single custom data type. Enums define a fixed set of named values.

- Array: uint[] public numbers;
- Mapping: mapping(address => uint) public balances;
- Struct: struct Student { string name; uint rollNo; }
- Enum: enum Status { Pending, Active, Completed }

8. Data Locations
Solidity uses storage, memory, and calldata as data locations. Storage is persistent blockchain storage. Memory is temporary and exists during function execution. Calldata is a read-only, non-persistent location mainly used for external function parameters and can be gas-efficient.

9. Ether and Wei
Ether (ETH) is the native currency of Ethereum. Wei is the smallest denomination of Ether. One Ether is equal to 10^18 Wei. Solidity allows Ether-denominated values to be used in payable functions and transactions.

10. Gas, Gas Price and Transactions
Gas represents the computational work required to execute an operation on the Ethereum blockchain. A gas price specifies how much is paid per unit of gas. A state-changing function creates a transaction and consumes gas. A read-only view function does not modify state and is normally executed as a call.

Transaction fee can be expressed as:

Transaction Fee = Gas Used × Gas Price

Tasks Performed

1. Opened Remix IDE and accessed the Solidity/LearnETH environment.
2. Created Solidity source files and wrote smart contract programs.
3. Compiled Solidity contracts using a compatible 0.8.x compiler.
4. Deployed contracts using Remix VM.
5. Executed functions and verified their outputs.
6. Practiced primitive data types, variables, pure/view functions, inputs and outputs.
7. Practiced visibility, constructors, modifiers, if-else statements and loops.
8. Practiced arrays, mappings, structs and enums.
9. Understood storage, memory and calldata data locations.
10. Practiced Ether/Wei and observed blockchain transactions and gas information.

Procedure / Steps

Step 1: Open Remix IDE

11. Open a web browser and visit Remix IDE.
12. Open the File Explorer from the left panel.
13. Create a new Solidity file, for example Lab7.sol.
14. Open the created file in the Solidity editor.

Step 2: Write the Solidity Program

The following program demonstrates a state-changing transaction using a mapping:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.18;

contract GasTransaction {
    mapping(address => uint) public balances;

    function setBalance(uint amount) public {
        balances[msg.sender] = amount;
    }

    function getBalance() public view returns (uint) {
        return balances[msg.sender];
    }
}
```

Step 3: Compile the Contract

15. Open the Solidity Compiler tab.
16. Select compiler version 0.8.18 or another compatible 0.8.x version.
17. Select the Solidity file containing the contract.
18. Click Compile.

19. Verify that there are no compilation errors.

Step 4: Deploy the Contract

20. Open Deploy & Run Transactions.
21. Select Remix VM as the environment.
22. Select GasTransaction from the contract dropdown.
23. Click Deploy.
24. Expand the Deployed Contracts section.

Step 5: Execute the State-Changing Function

25. Locate setBalance under Deployed Contracts.
26. Enter 500 as the amount.
27. Click transact.
28. A blockchain transaction is generated because the function modifies the mapping.

Step 6: Observe Gas and Transaction Details

29. Open the Terminal panel at the bottom of Remix.
30. Locate the transaction generated by setBalance(500).
31. Expand the transaction details if necessary.
32. Observe the transaction status, transaction hash, block number, gas used, execution cost, and other available details.
33. Take a screenshot of the Terminal showing the transaction information.

Step 7: Verify the Updated State

34. Click getBalance under the deployed contract.
35. The returned value should be 500.
36. This confirms that the state-changing transaction successfully updated the mapping.
37. Take a screenshot of the returned value.

Practical Screenshots

The screenshots from the submitted practical document are retained below as evidence of execution.

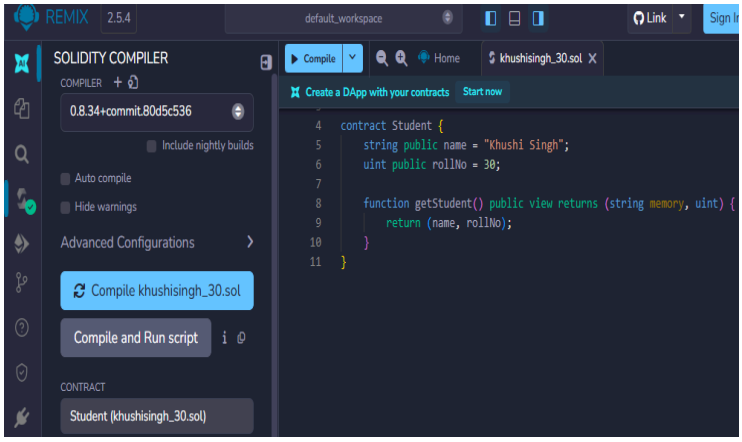


Figure 1: Practical screenshot

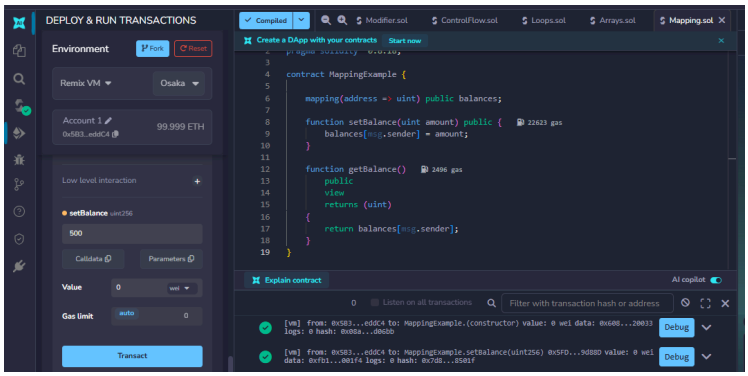


Figure 2: Practical screenshot

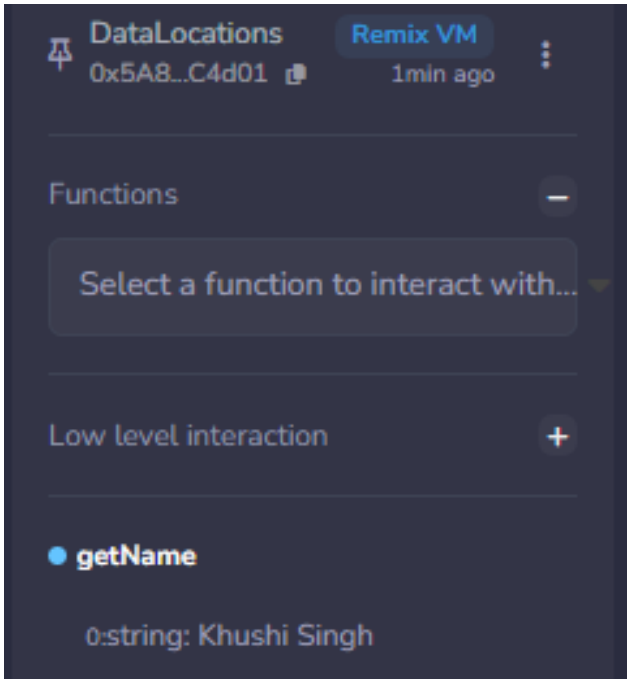


Figure 3: Practical screenshot

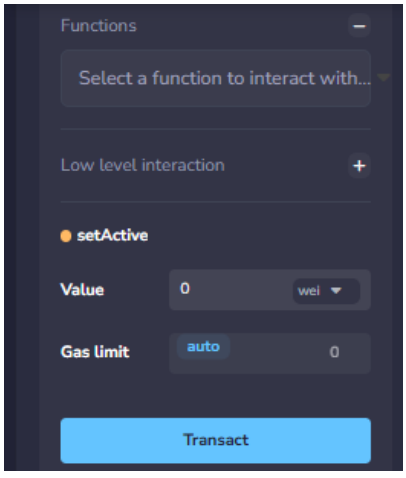


Figure 4: Practical screenshot

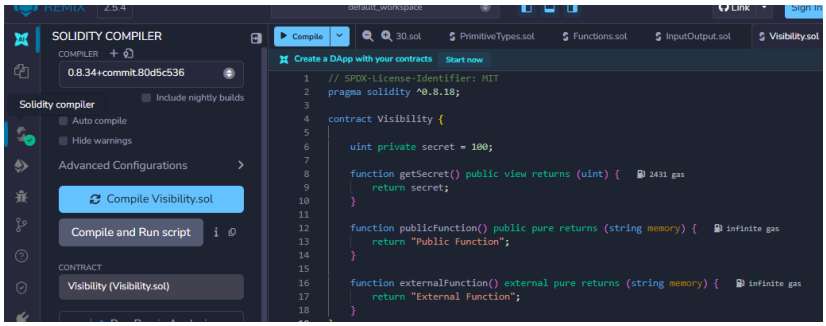


Figure 5: Practical screenshot

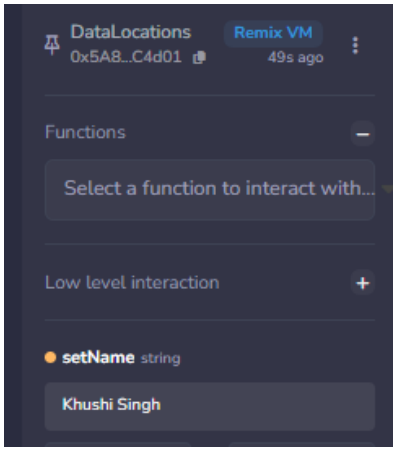


Figure 6: Practical screenshot

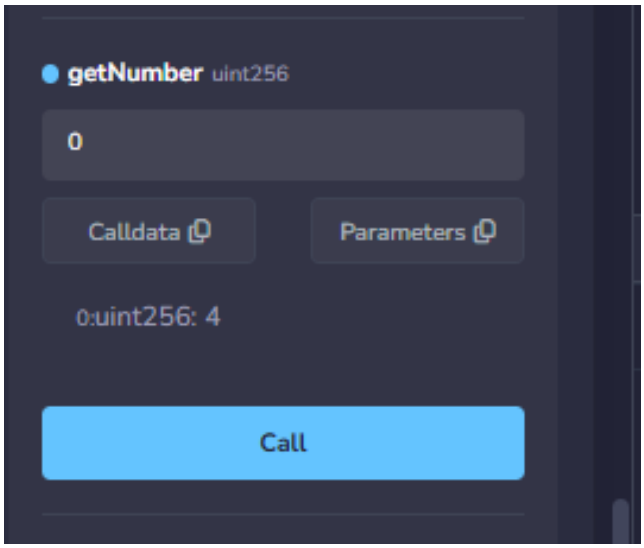


Figure 7: Practical screenshot

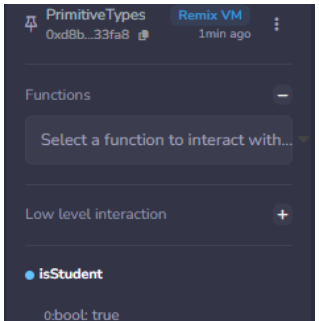


Figure 8: Practical screenshot

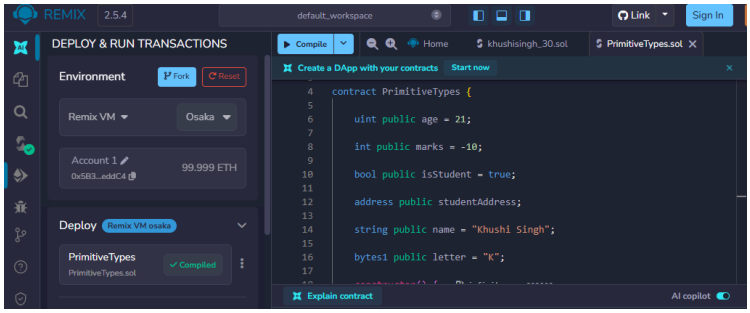


Figure 9: Practical screenshot

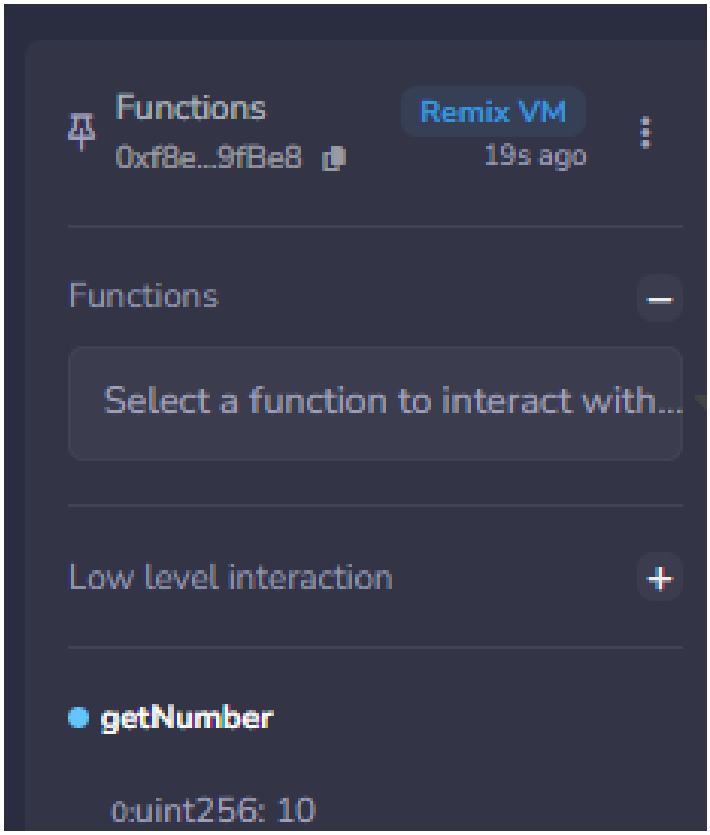


Figure 10: Practical screenshot

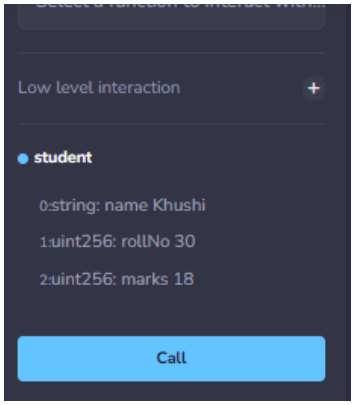


Figure 11: Practical screenshot

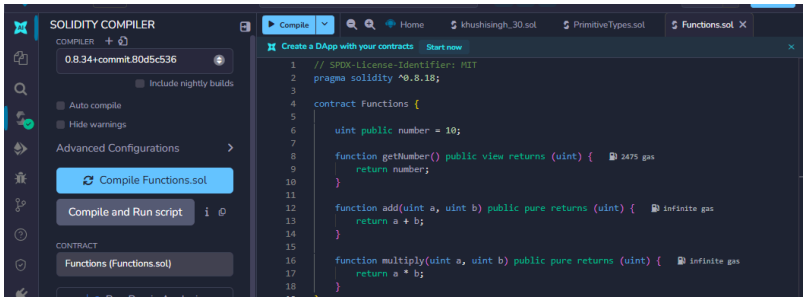


Figure 12: Practical screenshot

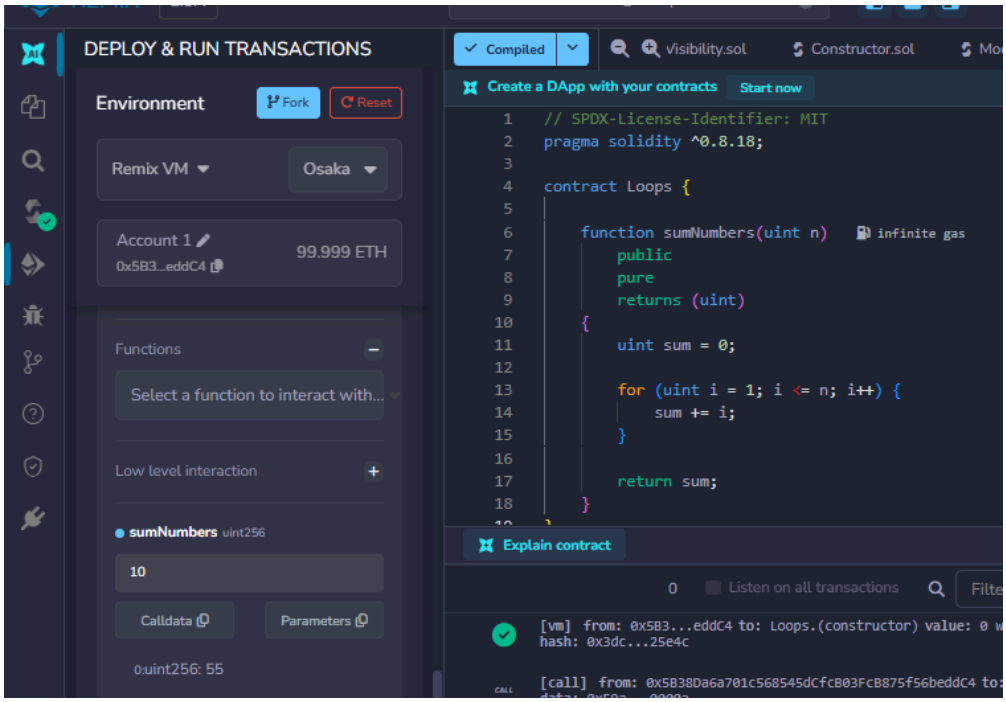


Figure 13: Practical screenshot

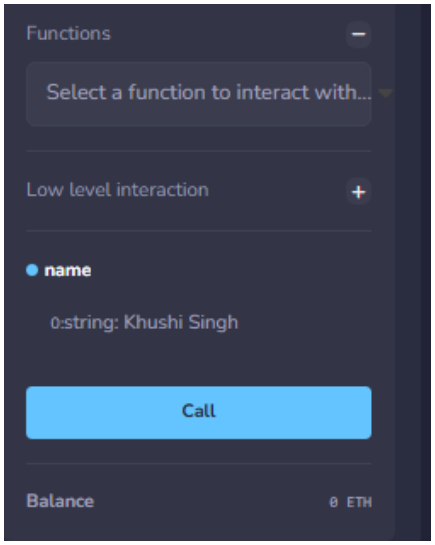


Figure 14: Practical screenshot

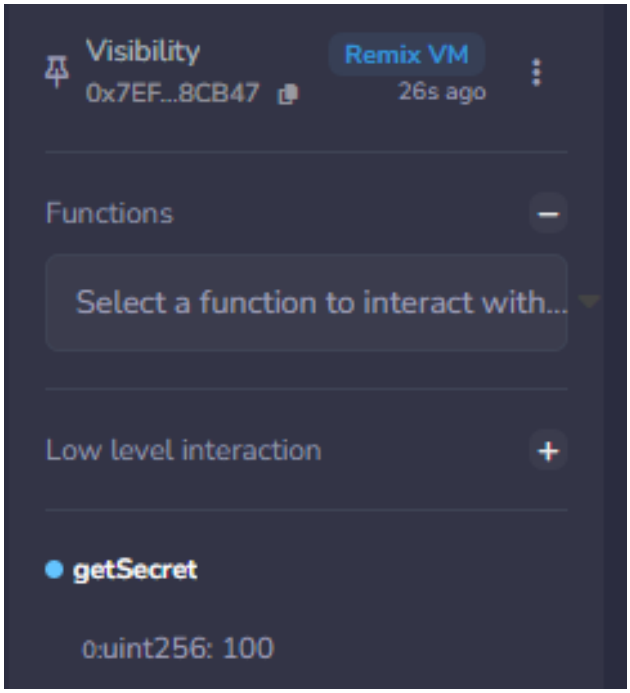


Figure 15: Practical screenshot

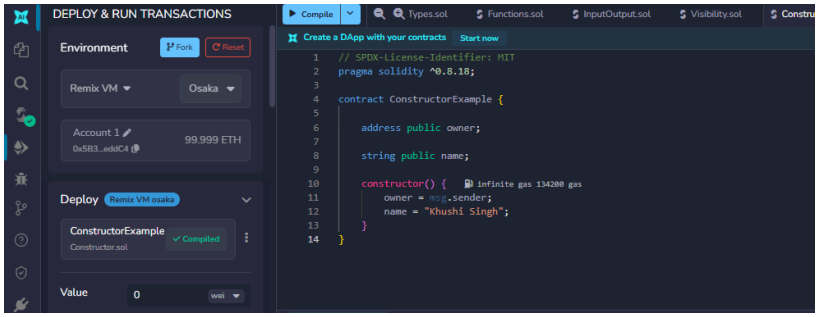


Figure 16: Practical screenshot

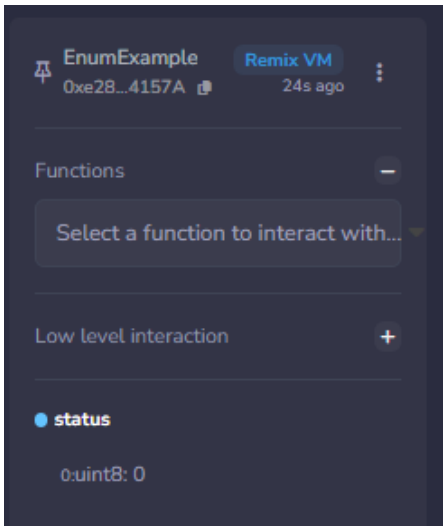


Figure 17: Practical screenshot

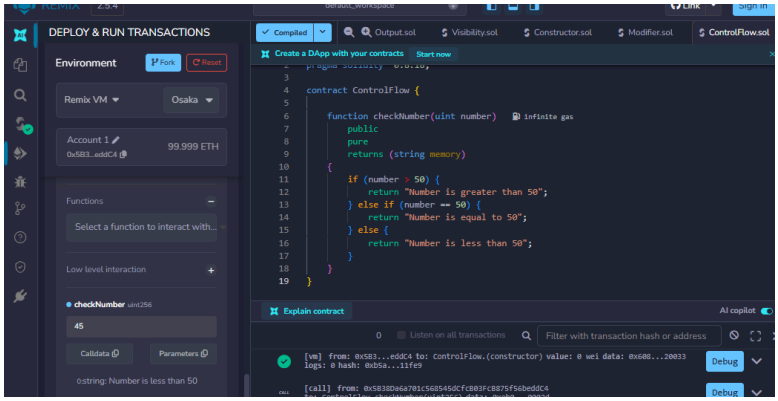


Figure 18: Practical screenshot

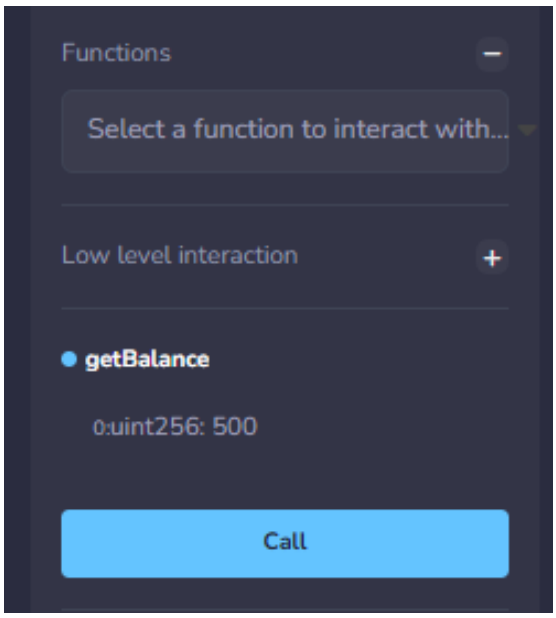


Figure 19: Practical screenshot

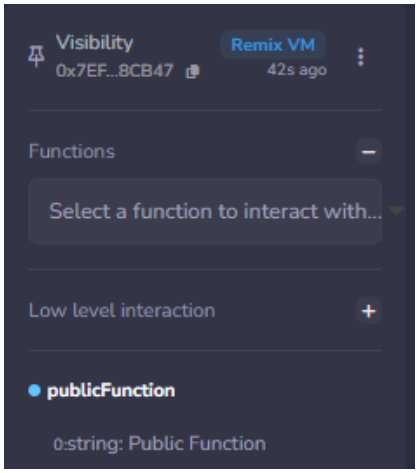


Figure 20: Practical screenshot

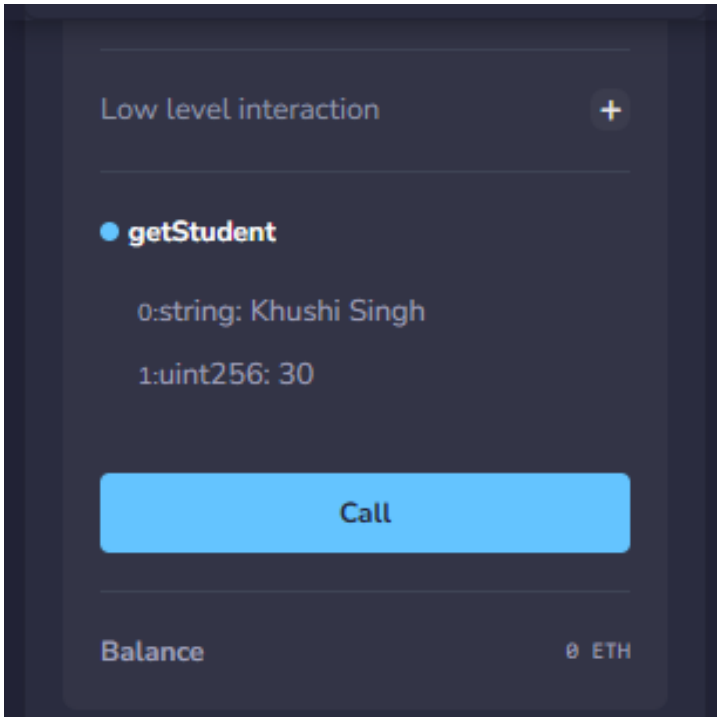


Figure 21: Practical screenshot

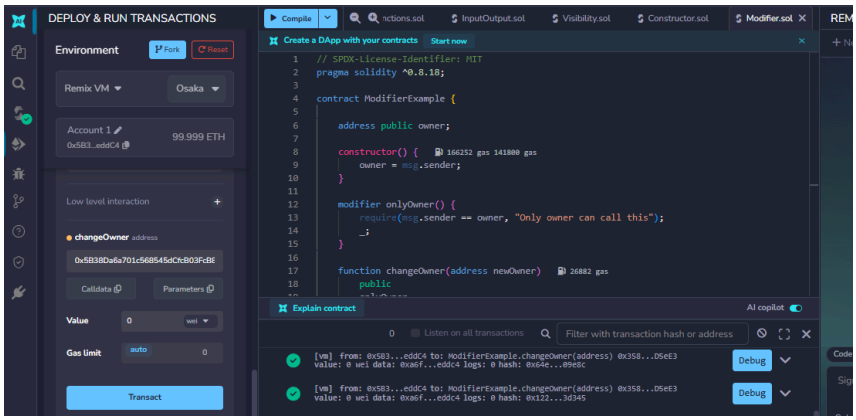


Figure 22: Practical screenshot

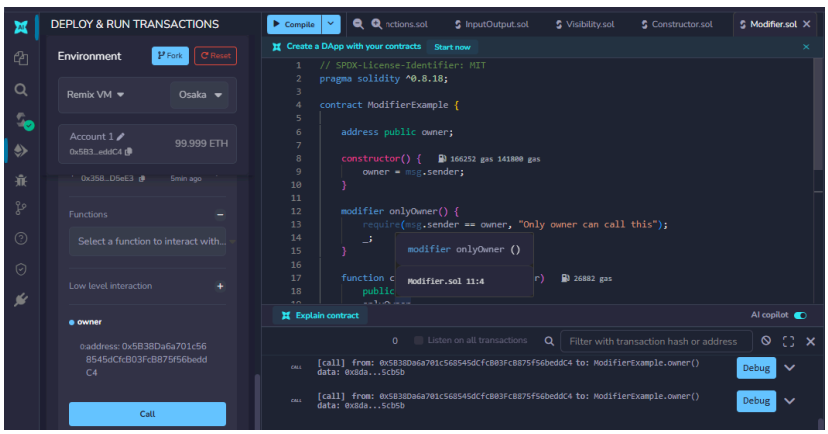


Figure 23: Practical screenshot

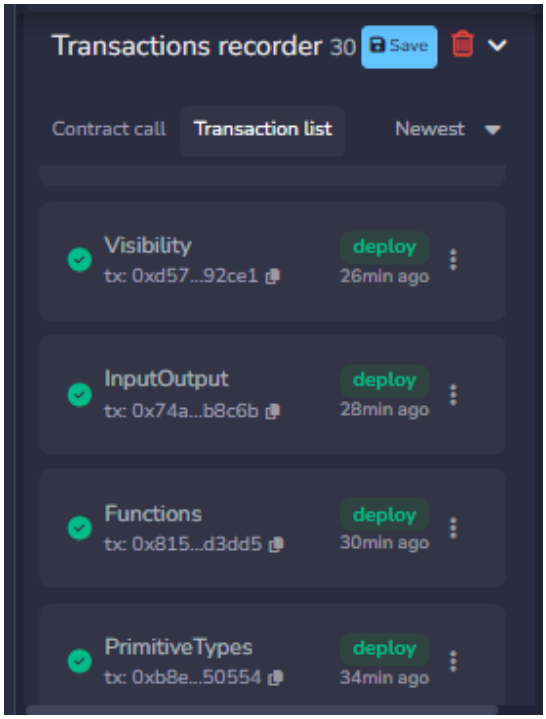


Figure 24: Practical screenshot

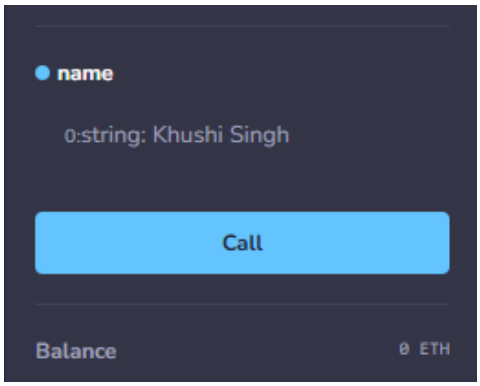


Figure 25: Practical screenshot

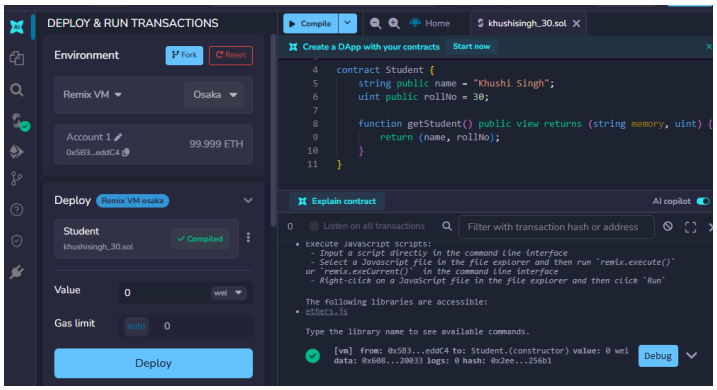


Figure 26: Practical screenshot

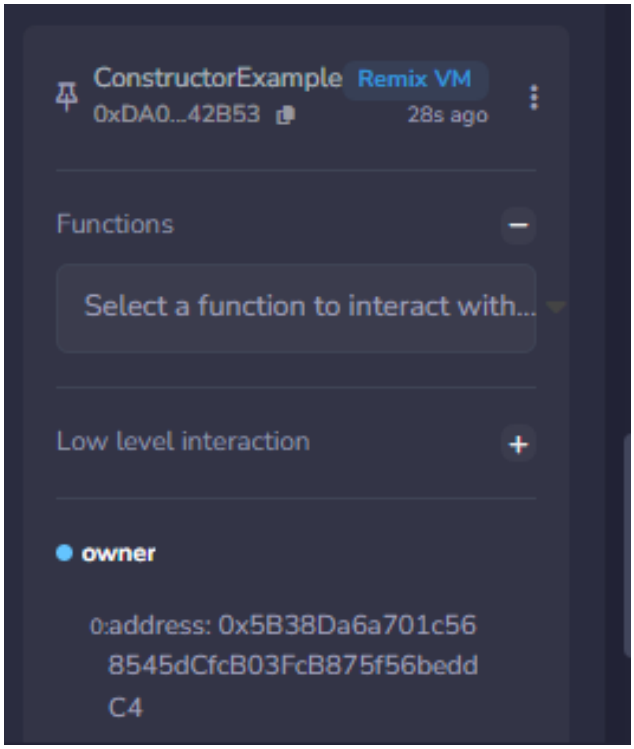


Figure 27: Practical screenshot

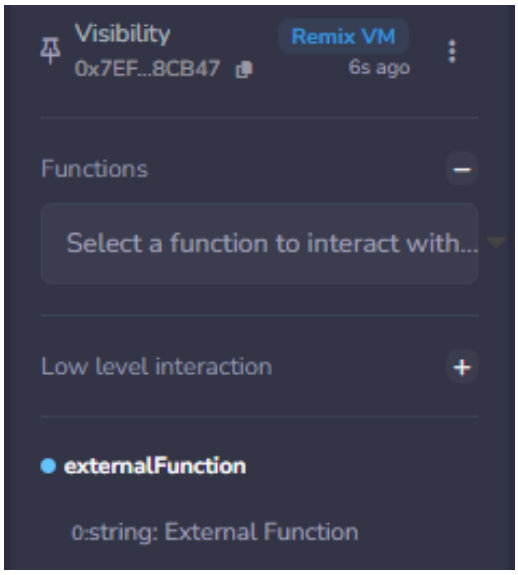
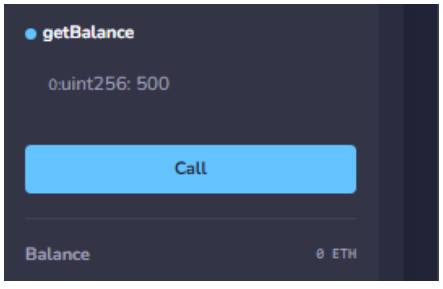


Figure 28: Practical screenshot



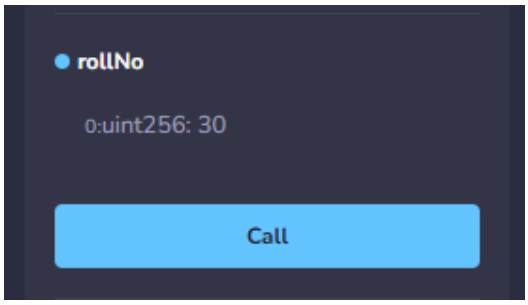


Figure 30: Practical screenshot

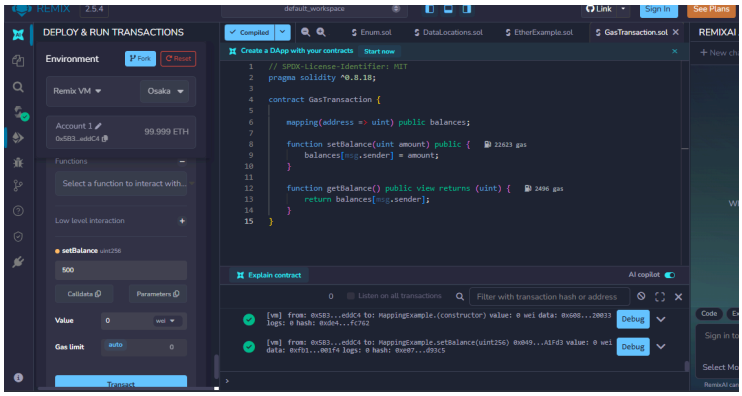
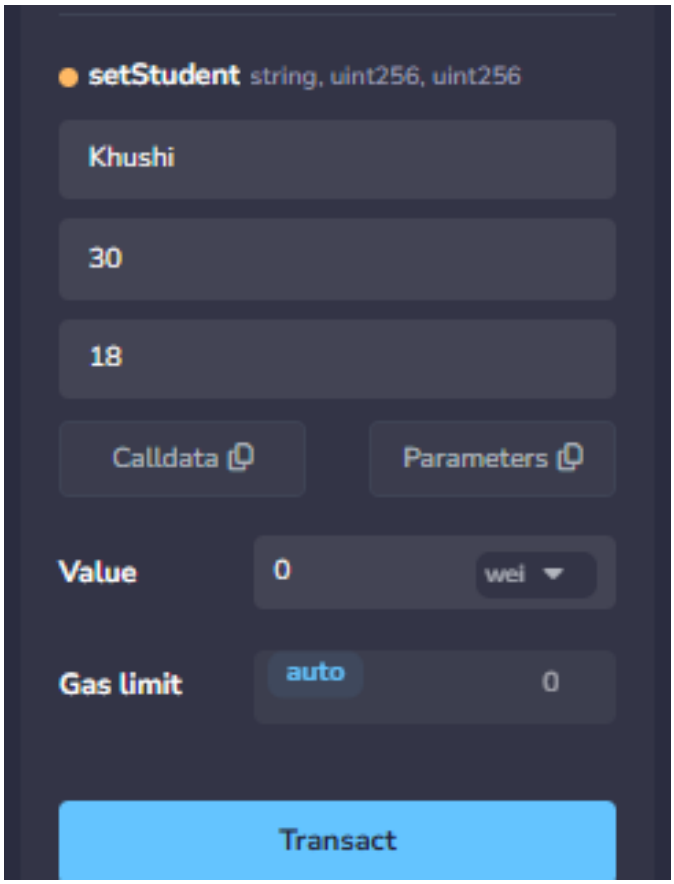


Figure 31: Practical screenshot



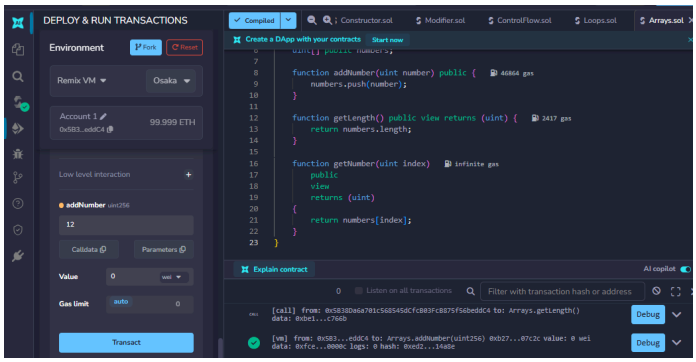


Figure 33: Practical screenshot

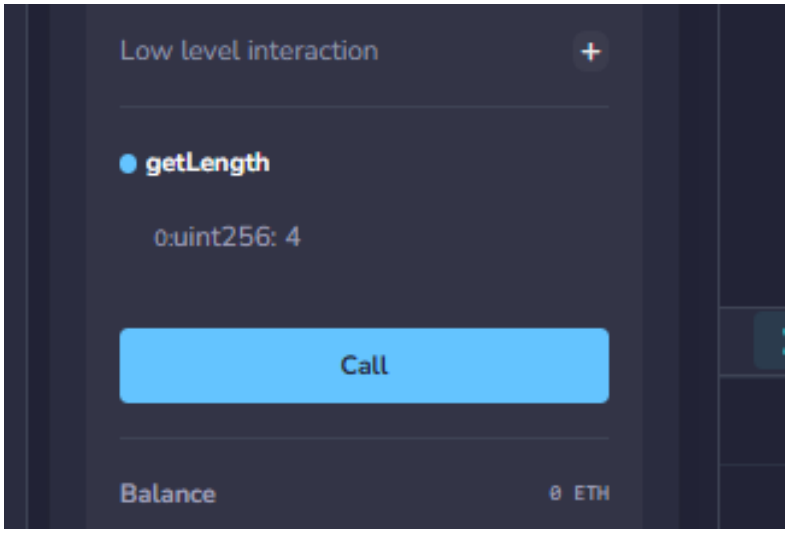


Figure 34: Practical screenshot

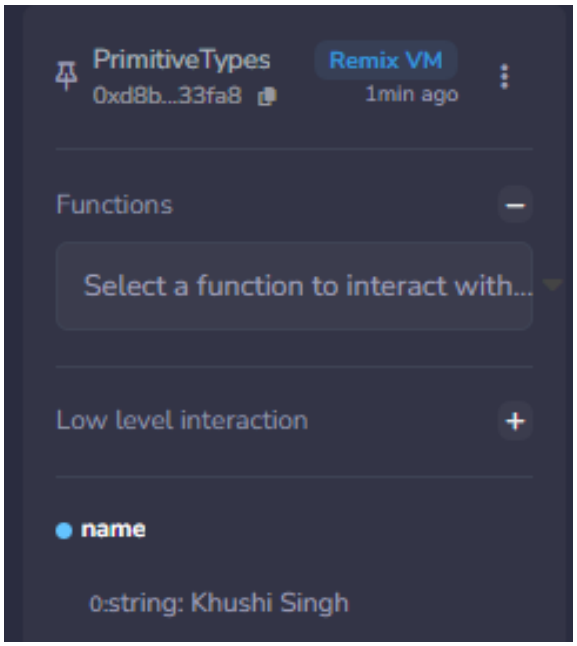


Figure 35: Practical screenshot

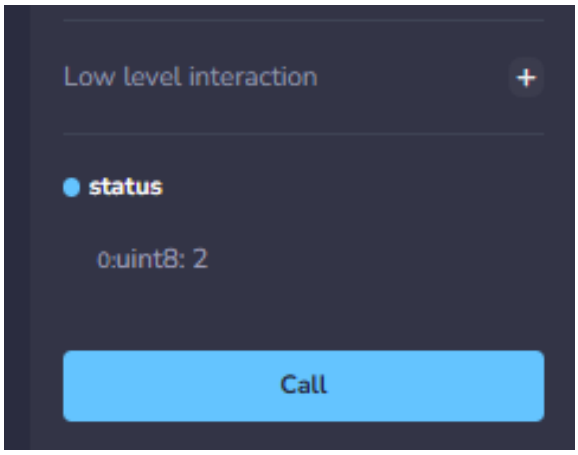


Figure 36: Practical screenshot

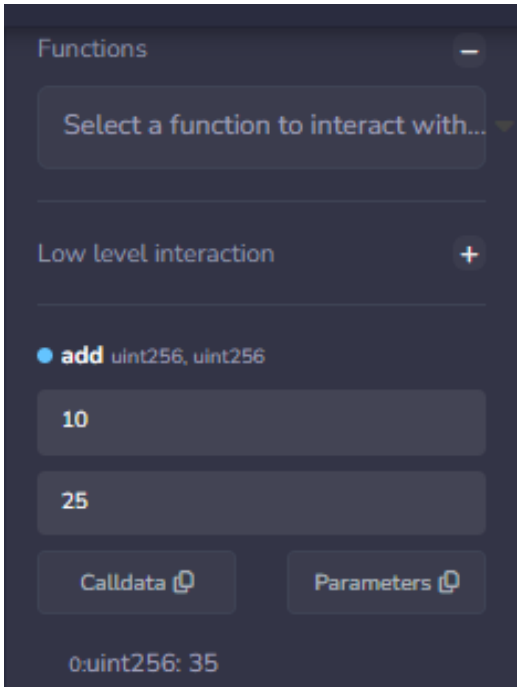


Figure 37: Practical screenshot

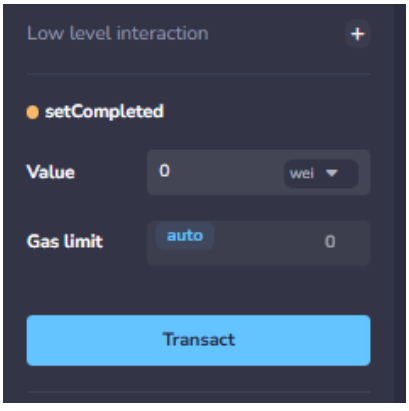


Figure 38: Practical screenshot

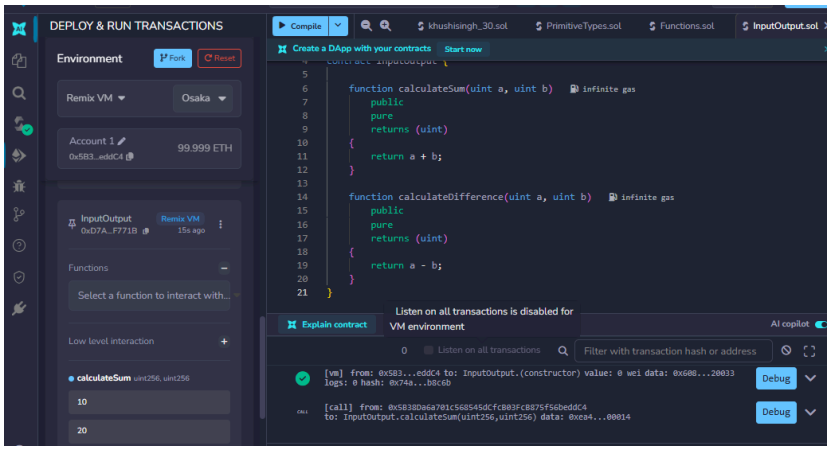


Figure 39: Practical screenshot

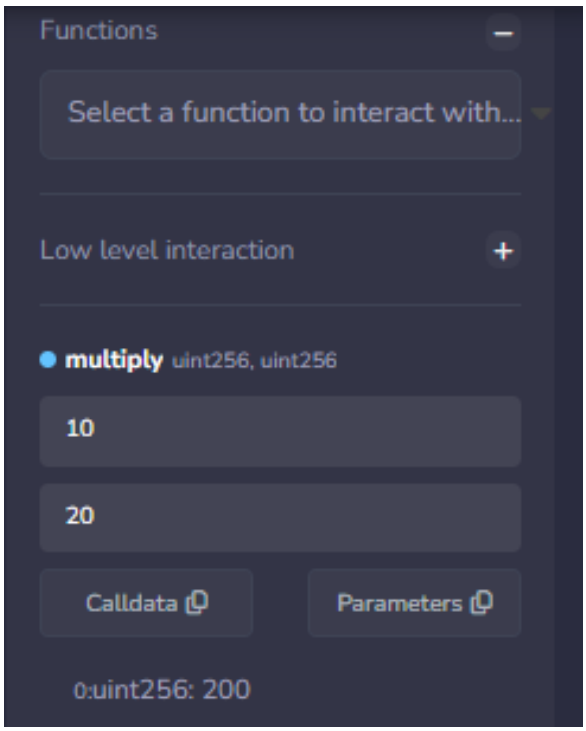


Figure 40: Practical screenshot

Conclusion

The experiment provided hands-on understanding of Solidity programming and smart contract development. The basic concepts of data types, variables, functions, visibility, constructors, modifiers, control flow, data structures, data locations, Ether, Wei, gas, and transactions were studied and practiced. The smart contract was successfully compiled, deployed, and executed in Remix VM, demonstrating how blockchain state changes are performed through transactions and how computational work is measured using gas.